Embedded System Design and IoT Project

Smart Autonomous Multi-Mode Robot Using Arduino

Project Team

R. Samshritha	2520030045
G. Lasya Sai	2520030538
P. Asritha	2520030592
P. Bindhu Madhavi	2520030077
G. Laxmisri	2520030264

B.Tech II Year

AY: 2026-27, Odd Semester

Supervised by

Dr. V. Ravi Kumar



Department of Computer Science Engineering

Koneru Lakshmaiah Education Foundation

Bachupally, Hyderabad

October, 2026

Certificate of Approval

The Department of Computer Science Engineering, Koneru Lakshmaiah Education Foundation, accepts this thesis titled **Smart Autonomous Multi-Mode Robot Using Arduino**, submitted by R. Samshritha (2520030045), G. Lasya Sai (2520030538), P. Asritha (2520030592), P. Bindhu Madhavi (2520030077), and G. Laxmisri (2520030264), in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor

Dr. V. Ravi Kumar

Signature: _____

Co-Supervisor

Dr. V. Dinesh Kumar

Signature: _____

Head of Department of Computer Science Engineering

Koneru Lakshmaiah Education Foundation, Bachupally, Hyderabad

Abstract

Mobile robots are widely used in warehouses, surveillance, education and assistive applications, yet many low-cost robot cars support only a single control mode. This project addresses that gap by designing and building a multi-function four-wheeled robot car on the Arduino UNO platform. The robot supports three operating modes: autonomous obstacle avoidance, smartphone control over Bluetooth, and voice control.

An HC-SR04 ultrasonic sensor mounted on an SG90 servo scans the surroundings, an HC-05 Bluetooth module receives commands from a smartphone application, and an L298N motor driver module drives four 60 RPM BO gear motors from a 7.4 V pack of two 18650 Li-ion cells. The firmware is written in embedded C++ (Arduino) using the Servo library. Each mode is implemented as a separate function and is selected with a switch, so the mode can be changed without re-uploading the code, and commands are exchanged as single-character messages over the serial port.

In the obstacle-avoidance mode the robot measures the distance ahead; when an obstacle is detected at 12 cm or less, it stops, reverses briefly, scans left and right, and turns toward the side with more free space. In the Bluetooth and voice modes, the characters F, B, L, R, S or the voice-command symbols are mapped to forward, backward, left, right and stop motions.

The design demonstrates that a low-cost embedded platform can integrate autonomous and remote operation in a single system, and that it can be extended with IoT features such as Wi-Fi connectivity and cloud monitoring. The report describes the hardware design, the circuit, the firmware logic, the test procedure and the possible improvements.

Keywords: Arduino UNO, Embedded System, Obstacle Avoidance, HC-SR04, HC-05 Bluetooth, L298N Motor Driver, Voice Control, IoT.

Contents

Introduction 5

Literature Survey 6

Proposed Work 8

Results and Discussion 20

Conclusion 23

Chapter 1

Introduction

1.1 Purpose of the Investigation

The purpose of this work is to design, build and evaluate a low-cost embedded robot that can drive itself around obstacles or be driven remotely from a smartphone, including by voice commands. The project applies the core ideas of an embedded-systems course: interfacing sensors and actuators to a microcontroller, writing real-time control firmware, and using a wireless link for remote operation, which is the first building block of an IoT device.

1.2 Problem Being Investigated

Many entry-level robot cars implement only one behaviour: they are either remote controlled or they avoid obstacles. A user who wants both must build and program separate robots. Integrating several modes on one microcontroller raises practical questions about pin allocation, sharing the serial port with the Bluetooth link, and structuring the code so each behaviour is easy to test.

1.3 Background and Importance

An embedded system is a computing device built into a larger product to perform a dedicated function. Robots, appliances and vehicles all rely on embedded systems that read sensors and drive actuators. When such devices are connected to a network, they become part of the Internet of Things (IoT).

Ultrasonic ranging is one of the most common ways for a low-cost robot to perceive obstacles. A transducer sends a short burst of 40 kHz sound and measures the time taken by the echo to return; the distance follows from the speed of sound. Motor driver modules based on the L298N dual H-bridge allow an Arduino to control DC motors without wiring individual driver chips. Hobby tutorials such as [2] have shown that these parts can be combined into a robot with obstacle avoidance, Bluetooth control and voice control, and the same approach is followed and documented in this report.

1.4 Thesis and General Approach

The thesis of this project is that a single Arduino UNO can host three control modes if each mode is written as an independent function and the Bluetooth module shares the hardware serial port. The approach has four steps: (1) select suitable low-cost components, (2) assemble and wire them according to the circuit diagram, (3) write modular firmware, and (4) test every mode separately and record the behaviour.

1.5 Criteria for Success

- The robot stops and avoids an obstacle that is detected at 12 cm or less.
- The robot responds to Bluetooth commands (F, B, L, R, S) without noticeable delay.
- The robot responds to voice commands mapped to the same movements.
- All four motors, the servo and both wireless and sensing parts operate reliably from the 7.4 V battery pack.

Chapter 2

Literature Survey

This chapter summarises the technologies and earlier practice that the project builds on.

2.1 Ultrasonic Obstacle Detection

The HC-SR04 module has a transmitter and a receiver. A 10 microsecond pulse on the Trig pin makes the module emit eight bursts of 40 kHz ultrasound; the Echo pin then goes high for as long as the echo takes to return. With the speed of sound at roughly 340 m/s, one centimetre of distance takes about 29 microseconds, so the distance in centimetres is the echo time divided by 29 and then by 2 (the pulse travels to the object and back). It works from 2 cm to about 400 cm but reads poorly on soft or strongly angled surfaces. Mounting the sensor on a servo lets a robot look left and right and choose a free direction [2].

2.2 Motor Control with the L298N Driver

A DC motor needs more current than a microcontroller pin can supply and must be reversible. The L298N contains two H-bridges that provide this, so one module can drive two motor channels. The direction of each channel is set with two input pins (IN1, IN2 and IN3, IN4), and the enable pins ENA and ENB either carry a PWM signal for speed control or, when their jumpers are fitted, keep the channel always enabled at full voltage. Motors on the same side of the robot can be connected in parallel to one channel, so one L298N is enough for a four-wheel-drive robot [5].

2.3 Bluetooth Communication

The HC-05 module implements the Bluetooth serial port profile. To the microcontroller it looks like a normal UART at 9600 baud by default, so a character sent from a phone application arrives in the Arduino's serial buffer and can be read with Serial.read(). It shares the RX and TX pins with USB, so it must be unplugged while uploading; indoor range is about 10 m.

2.4 Voice Control

Voice control in hobby robots does not need speech-processing hardware on the robot. A smartphone application recognises the phrase and sends one character over Bluetooth, so the firmware handles voice commands like button commands with a different set of symbols [2].

2.5 Comparison of Approaches

Area	Technique	Advantage	Limitation
Obstacle avoidance	HC-SR04 ultrasonic sensor on a scanning servo	Low cost, simple, wide field of view	Poor on soft or angled surfaces; scan adds delay
Remote control	HC-05 Bluetooth with smartphone app	Easy serial interface, no infrastructure	Short range (about 10 m)
Voice control	Phone speech recognition sending characters	No extra hardware on the robot	Depends on noise and recognition accuracy

Area	Technique	Advantage	Limitation
Motor driving	L298N dual H-bridge module	Drives four motors as two pairs; simple direction inputs	Voltage drop across the bridge; limited current

2.6 Research Gap

Most published and hobby designs implement a single control mode. This project integrates autonomous, Bluetooth and voice control on one low-cost platform with modular firmware, and documents the wiring, firmware and testing so that it can be reproduced.

2.7 Design Requirements Drawn from the Survey

The survey leads to the following requirements for the robot designed in Chapter 3:

- Use a scanning ultrasonic sensor so that the robot can compare free space on both sides before turning.
- Use a dual H-bridge driver module (L298N) so that the four motors can be controlled as a left pair and a right pair with four digital pins.
- Use a UART Bluetooth module so that both button and voice commands can share one simple character protocol.
- Keep each mode in its own function and select it with a switch so that it can be tested, modified and explained independently.
- Power the whole robot from a single rechargeable 7.4 V pack with an on/off switch.

These requirements are met by the components and firmware structure described in the next chapter.

Chapter 3

Proposed Work

3.1 Block Diagram

The system is made of four groups of blocks: the power supply, the controller, the input devices (ultrasonic sensor and Bluetooth module) and the actuators (four 60 RPM BO gear motors and a servo). Figure 3.1 shows how they are connected.

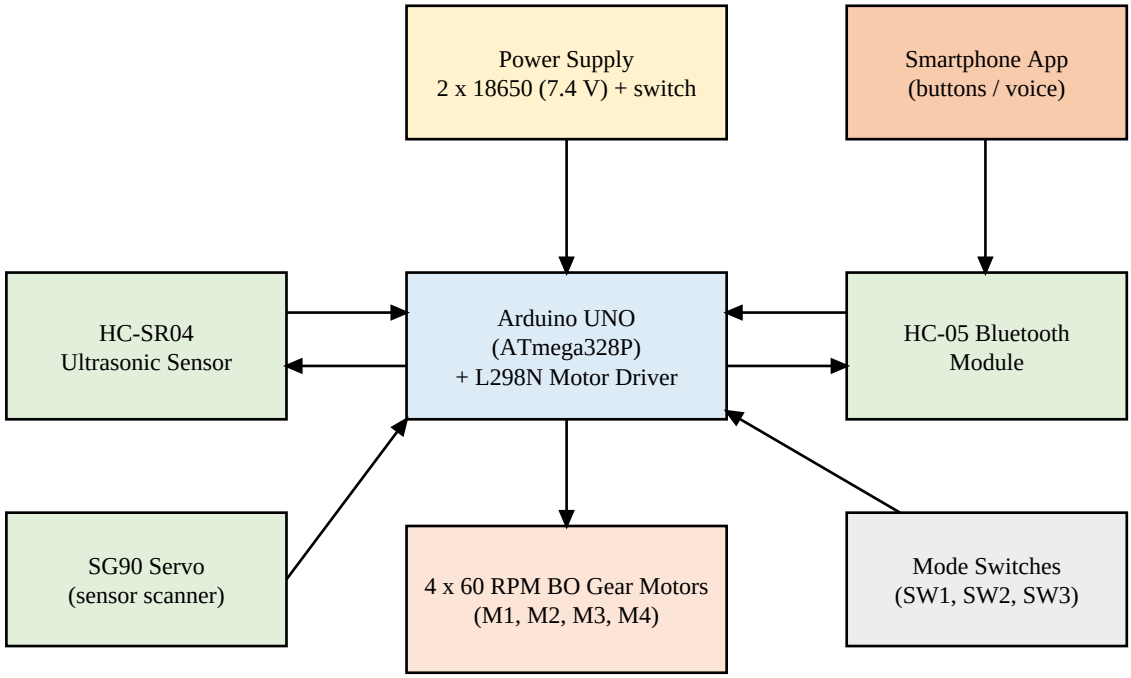


Figure 3.1: Block diagram of the multi-function robot

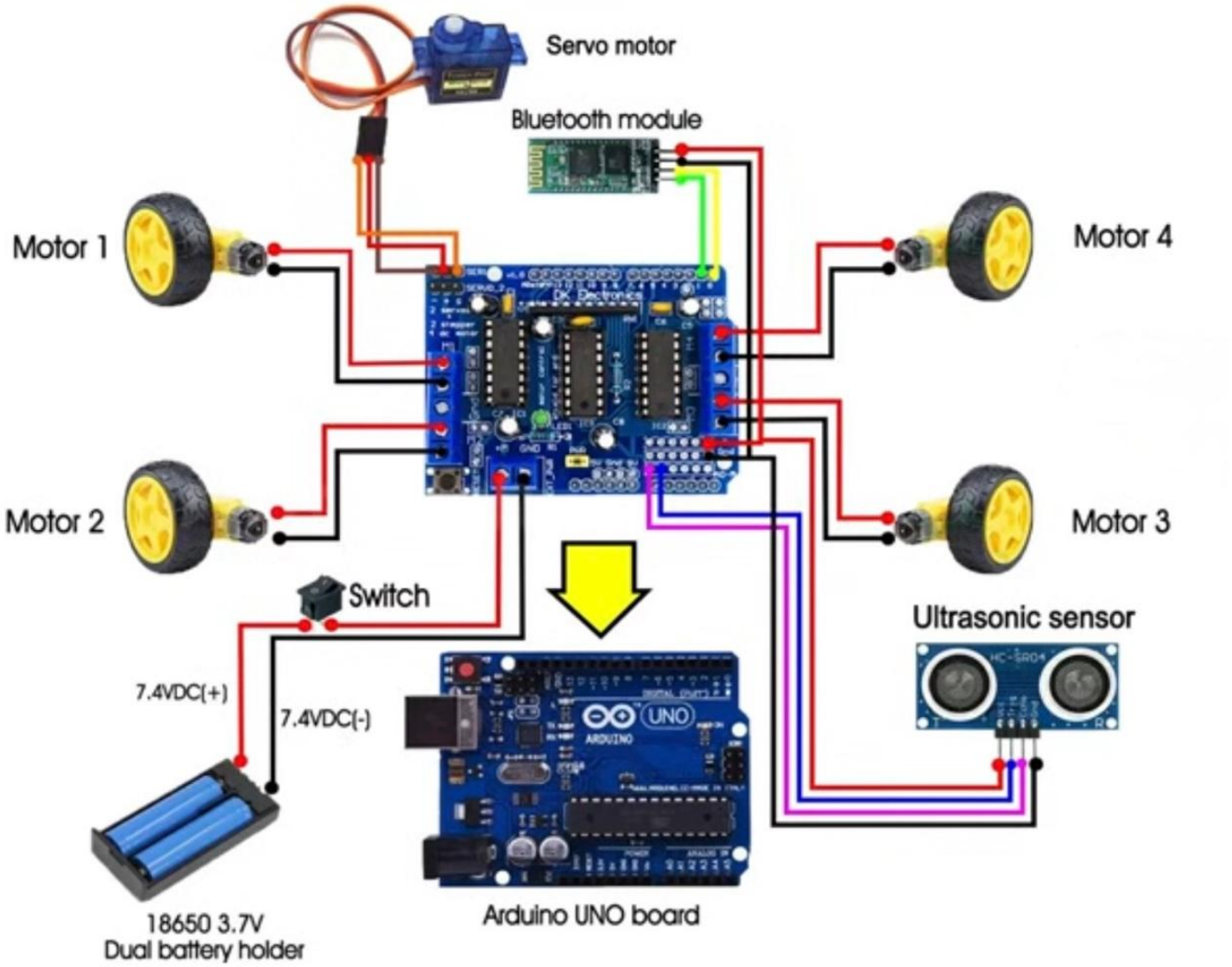
Note on the drive motors: The four actuators in the block diagram are 60 RPM BO (battery-operated) gear motors, each fitted with a wheel. BO gear motors are lightweight and compact, and they give high torque at low speed, which suits a small four-wheel-drive robot. They run directly from the 7.4 V supply through the L298N motor driver (outputs OUT1 to OUT4).

3.2 Components Used

Component	Qty	Role in the project
Arduino UNO (ATmega328P)	1	Main controller running the firmware
L298N motor driver module	1	Drives the four DC motors as left and right pairs
HC-SR04 ultrasonic sensor	1	Measures distance to obstacles
HC-05 Bluetooth module	1	Wireless link to the smartphone
SG90 micro servo	1	Rotates the ultrasonic sensor left and right
60 RPM BO gear motor with wheel	4	Four-wheel drive

Robot Hardware Connection Diagram

Smart Autonomous Multi-Mode Robot Using Arduino



Component	Qty	Role in the project
18650 Li-ion cell, dual holder	2	7.4 V supply (2 x 3.7 V in series)
Mode selection switches (SW1 to SW3)	3	Select obstacle, Bluetooth or voice mode
Slide switch, jumper wires	-	Power switch and connections
Foam board / cardboard	-	Chassis

3.3 Circuit Diagram

Figure 3.2 shows the connections of the robot. The two 18650 cells are connected through a switch to the supply terminal of the L298N motor driver. The left-side motors (M1, M2) are wired to the first output channel (OUT1, OUT2) and the right-side motors (M3, M4) to the second channel (OUT3, OUT4). The driver inputs IN1 to IN4 are connected to Arduino pins 8 to 11. The servo signal wire is connected to A2, the HC-SR04 is wired to the analog pins A0 (Echo) and A1 (Trig) with 5 V and ground, and the HC-05 is connected to the RX/TX pins and power pins. Three mode switches are connected between pins 2, 3, 4 and ground.

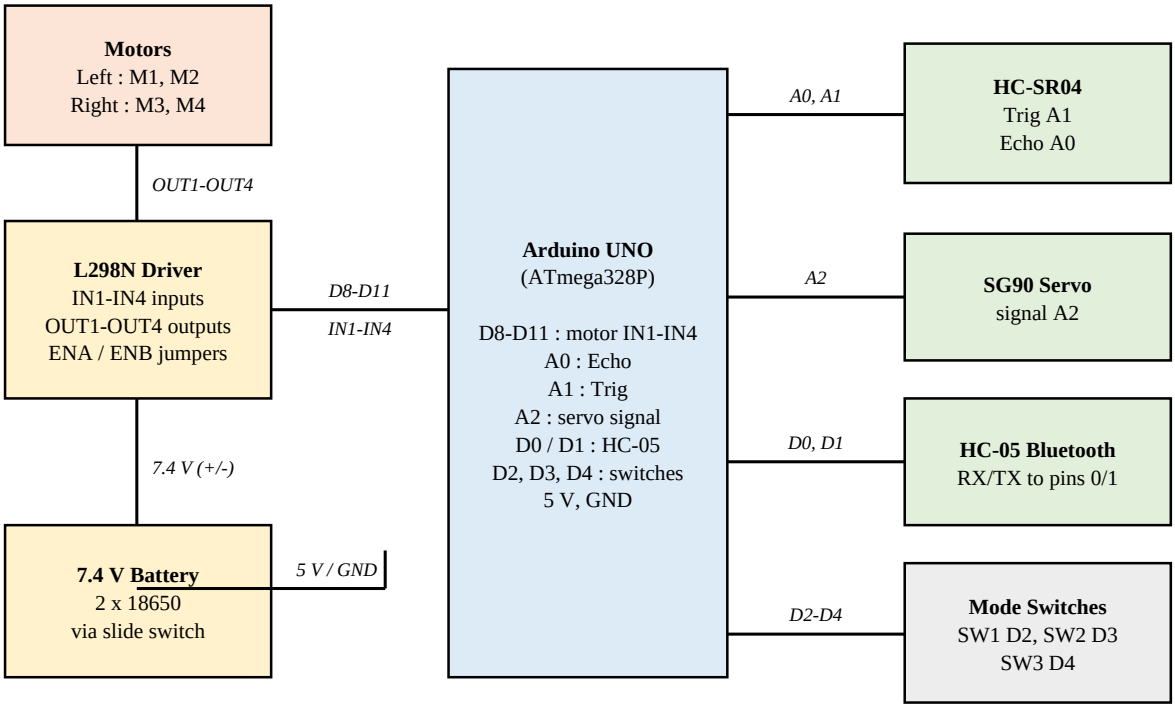


Figure 3.2: Connection diagram of the robot (Arduino UNO with L298N driver, four motors, servo, HC-05, HC-SR04 and mode switches)

3.4 Pin Allocation

Device	Arduino pin / port	Notes
HC-SR04 Trig	A1	Digital output, 10 microsecond pulse
HC-SR04 Echo	A0	Digital input read with pulseIn() (30 ms time-out)

Device	Arduino pin / port	Notes
SG90 servo signal	A2	Positions: 20 (right), 103 (centre), 180 (left)
L298N IN1, IN2	D8, D9	Left-side motors M1, M2 (OUT1, OUT2)
L298N IN3, IN4	D10, D11	Right-side motors M3, M4 (OUT3, OUT4)
HC-05 TXD / RXD	RX (pin 0) / TX (pin 1)	UART at 9600 baud; unplug during upload
Mode switch SW1	D2	Obstacle-avoidance mode (ON = LOW, internal pull-up)
Mode switch SW2	D3	Bluetooth control mode (ON = LOW, internal pull-up)
Mode switch SW3	D4	Voice control mode (ON = LOW, internal pull-up)
Motors 1 to 4	OUT1 to OUT4 of L298N	60 RPM BO gear motors, in two pairs (left and right)
Battery 7.4 V	L298N supply terminal	Through a slide switch

The ENA and ENB pins of the L298N are left on their jumpers, so both motor channels are always enabled and the motors run at the full supply voltage; speed is not set by PWM. The Speed constant in the sketch is therefore kept only for reference.

3.5 Working Principle of the Modes

Mode selection. The mode is chosen with three switches on pins 2, 3 and 4, which are configured with the internal pull-up resistors. In loop(), the firmware checks the switches and runs the function of the switch that is ON (LOW): Obstacle() for SW1, Bluetoothcontrol() for SW2 and voicecontrol() for SW3. Only one switch should be ON at a time.

Obstacle avoidance. The firmware triggers the ultrasonic sensor continuously and prints the distance on the serial port. If no echo is received within 30 ms, the distance is taken as 400 cm. If the measured distance is greater than 12 cm, the motors on both sides run forward. When the distance is 12 cm or less, the robot stops, reverses for 100 ms, stops for 200 ms, and then turns the servo to 180 degrees (left) and to 20 degrees (right) to measure the free distance on each side, returning the servo to the centre position (103 degrees) after each scan. The robot then turns toward the side with the larger distance for 500 ms and stops for 200 ms. If both distances are equal, it simply reverses for 500 ms and stops. After this the loop resumes.

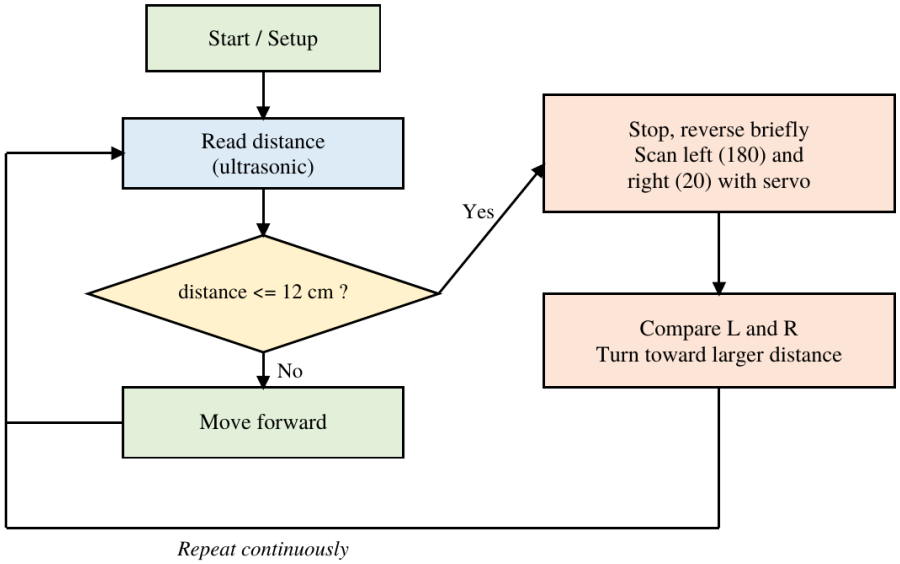


Figure 3.3: Flowchart of the obstacle-avoidance mode

Bluetooth control. The smartphone application sends one character for every button press: F (forward), B (backward), L (left), R (right) and S (stop). The firmware reads the character from the serial buffer and calls the matching movement function. The last received command stays active until a new command arrives.

Voice control. The application converts each spoken phrase into a symbol: ^ (forward), - (backward), < (left), > (right) and * (stop). For the left and right commands the robot first scans that side with the servo and turns only if at least 10 cm of free space is measured, which protects the robot from turning into a wall.

Turning method. The robot has no steering mechanism; it turns like a tank. To turn right, the motors on one side (M1, M2) run backward while the other side (M3, M4) runs forward, and the reverse for turning left. This is done by setting the direction inputs IN1 to IN4 of the L298N.

3.6 Methodology

1. **Step 1 - Chassis and mounting.** Cut the foam board, glue the four 60 RPM BO gear motors to it, and fix the Arduino, the L298N driver and the battery holder on it.
2. **Step 2 - Wiring.** Connect the motor terminals, the servo, the HC-SR04, the HC-05, the three mode switches and the battery according to Figure 3.2 and Section 3.4.
3. **Step 3 - Firmware.** Write the sketch of Section 3.7 in the Arduino IDE (only the Servo library, which is built in, is needed).
4. **Step 4 - Uploading.** Disconnect the RX and TX wires of the HC-05, upload the sketch over USB, then reconnect the wires.
5. **Step 5 - Application set-up.** Install the Bluetooth control or voice control application on the phone, pair it with the HC-05 and configure the command characters.
6. **Step 6 - Testing.** Turn on one mode switch at a time, test each mode separately, as described in Chapter 4, and record the observations.

3.7 Firmware

The complete firmware used on the robot is given below. All three mode functions are present in the same sketch, and the mode switches on pins 2, 3 and 4 decide which one runs, so the code does not have to be changed or re-uploaded to change the mode.

```
/*
   Obstacle Avoiding + Bluetooth Control + Voice Control Robot
   Arduino UNO + L298N + HC-SR04 + Servo + HC-05
*/

#include <Servo.h>

// ---------------- PIN DEFINITIONS ----------------

int sw1 = 2;
int sw2 = 3;
int sw3 = 4;

// Ultrasonic sensor
#define Echo A0
#define Trig A1

// Servo
#define SERVO_PIN A2

// L298N Motor Driver
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

// Motor speed is controlled by L298N ENA/ENB jumpers
#define Speed 170

// Servo center position
#define spoint 103

// ---------------- VARIABLES ----------------

char value;

int distance;
int Left;
int Right;

int L = 0;
int R = 0;
int L1 = 0;
int R1 = 0;

Servo servo;

// ---------------- SETUP ----------------
```

```
void setup()
{
  Serial.begin(9600);

  pinMode(sw1, INPUT_PULLUP);pinMode(sw2, INPUT_PULLUP);pinMode(sw3, INPUT_PULLUP);
  // Ultrasonic
  pinMode(Trig, OUTPUT);
  pinMode(Echo, INPUT);

  // Motor pins
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  // Servo
  servo.attach(SERVO_PIN);
  servo.write(spoint);

  Stop();
}

// ---------------- MAIN LOOP ----------------

void loop()
{
  if(digitalRead(sw1) == LOW)
    {
     Obstacle();
    }

  // Uncomment if Bluetooth manual control is required
  if(digitalRead(sw2) == LOW)
    {
     Bluetoothcontrol();
    }
  // Uncomment if voice control is required

  if(digitalRead(sw3) == LOW)
    {
     voicecontrol();
    }
}

// ================================================
// BLUETOOTH CONTROL
// ================================================
```

```
void Bluetoothcontrol()
{
  if (Serial.available() > 0)
  {
    value = Serial.read();
    Serial.println(value);
  }

  if (value == 'F')
  {
    forward();
  }
  else if (value == 'B')
  {
    backward();
  }
  else if (value == 'L')
  {
    left();
  }
  else if (value == 'R')
  {
    right();
  }
  else if (value == 'S')
  {
    Stop();
  }
}

// ================================================
// OBSTACLE AVOIDANCE
// ================================================

void Obstacle()
{
  distance = ultrasonic();

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  if (distance <= 12)
  {
    Stop();

    // Move slightly backward
    backward();
    delay(100);
```

```
    Stop();
    delay(200);

    // Look left
    L = leftsee();

    servo.write(spoint);
    delay(300);

    // Look right
    R = rightsee();

    servo.write(spoint);
    delay(300);

    Serial.print("Left: ");
    Serial.print(L);

    Serial.print("  Right: ");
    Serial.println(R);

    if (L < R)
    {
      // More space on right
      right();
      delay(500);
      Stop();
      delay(200);
    }
    else if (L > R)
    {
      // More space on left
      left();
      delay(500);
      Stop();
      delay(200);
    }
    else
    {
      // Both sides similar
      backward();
      delay(500);
      Stop();
    }
  }
  else
  {
    forward();
  }
}
```

```
// ================================================
// VOICE CONTROL
// ================================================

void voicecontrol()
{
  if (Serial.available() > 0)
  {
    value = Serial.read();

    Serial.println(value);

    if (value == '^')
    {
      forward();
    }
    else if (value == '-')
    {
      backward();
    }
    else if (value == '<')
    {
      L = leftsee();

      servo.write(spoint);

      if (L >= 10)
      {
        left();
        delay(500);
        Stop();
      }
      else
      {
        Stop();
      }
    }
    else if (value == '>')
    {
      R = rightsee();

      servo.write(spoint);

      if (R >= 10)
      {
        right();
        delay(500);
        Stop();
      }
      else
      {
        Stop();
      }
    }
    else if (value == '*')
    {
      Stop();
    }
  }
}
```

```
// ================================================
// ULTRASONIC SENSOR
// ================================================

int ultrasonic()
{
  digitalWrite(Trig, LOW);
  delayMicroseconds(4);

  digitalWrite(Trig, HIGH);
  delayMicroseconds(10);

  digitalWrite(Trig, LOW);

  long t = pulseIn(Echo, HIGH, 30000);

  if (t == 0)
  {
    return 400;
  }

  long cm = t / 29 / 2;

  return cm;
}

// ================================================
// MOTOR CONTROL
// ================================================

// Forward
void forward()
{
  digitalWrite(IN1, HIGH);digitalWrite(IN2, LOW);

  digitalWrite(IN3, HIGH);digitalWrite(IN4, LOW);
}

// Backward
void backward()
{
  digitalWrite(IN1, LOW);digitalWrite(IN2, HIGH);

  digitalWrite(IN3, LOW);digitalWrite(IN4, HIGH);
}

// Right turn
void right()
{
  digitalWrite(IN1, LOW);digitalWrite(IN2, HIGH);
```

```
  digitalWrite(IN3, HIGH);digitalWrite(IN4, LOW);
}

// Left turn
void left()
{
  digitalWrite(IN1, HIGH);digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);digitalWrite(IN4, HIGH);
}

// Stop
void Stop()
{
  digitalWrite(IN1, LOW);digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);digitalWrite(IN4, LOW);
}

// ================================================
// SERVO LOOK RIGHT
// ================================================

int rightsee()
{
  servo.write(20);
  delay(800);

  Right = ultrasonic();

  return Right;
}

// ================================================
// SERVO LOOK LEFT
// ================================================

int leftsee()
{
  servo.write(180);
  delay(800);

  Left = ultrasonic();

  return Left;
}
```

3.8 Explanation of the Code

- **Library and constants:** Servo.h controls the scanning servo. The sensor pins (A0, A1), servo pin (A2), driver pins (8 to 11), servo centre (103 degrees) and the Speed value (170, kept for reference) are defined at the top of the sketch.
- **setup():** starts the serial port at 9600 baud (shared with the HC-05), sets the three switch pins as INPUT_PULLUP, sets the Trig and Echo pin directions, sets the four driver pins as outputs, attaches the servo, centres it and stops the motors.
- **loop():** reads the three mode switches and calls Obstacle(), Bluetoothcontrol() or voicecontrol() for the switch that is LOW (ON).
- **ultrasonic():** sends a 10 microsecond trigger pulse, measures the echo width with pulseIn() using a 30 ms time-out and converts it to centimetres by dividing by 29 and by 2. If no echo returns it reports 400 cm.
- **Movement functions:** forward(), backward(), right(), left() and Stop() set the direction pins IN1 to IN4 of the L298N with digitalWrite(); Stop() sets all four LOW so the motors stop.

- **leftsee() / rightsee():** turn the servo to 180 or 20 degrees, wait 800 ms for it to settle and return the measured distance.
- **Mode functions:** Obstacle(), Bluetoothcontrol() and voicecontrol() implement the three behaviours described in Section 3.5.

Chapter 4

Results and Discussion

4.1 Test Set-up

The finished robot was tested on a flat, hard indoor surface. The obstacle test used a cardboard box placed in front of the robot, the Bluetooth and voice tests used a smartphone paired with the HC-05 over a distance of a few metres, and sensor readings were compared with a measuring tape. The robot was powered from the 7.4 V pack, and one mode switch was turned on at a time. Each test was repeated several times and the behaviour was noted.

4.2 Functional Test Plan and Observations

Table 4.1 lists each test, the behaviour expected from the firmware and the behaviour observed on the robot.

No.	Mode	Test	Expected behaviour	Observed
1	Obstacle	Clear path ahead	Robot drives forward	Robot drove forward continuously. Pass
2	Obstacle	Box placed at 12 cm or less	Stops, reverses briefly, scans both sides	Robot stopped, backed up slightly and scanned left and right. Pass
3	Obstacle	Left side more open	Robot turns left and continues	Robot turned left and moved on. Pass
4	Obstacle	Right side more open	Robot turns right and continues	Robot turned right and moved on. Pass
5	Obstacle	Both sides at equal distance	Robot reverses and stops	Robot reversed for about half a second and stopped. Pass
6	Bluetooth	Press F / B	Moves forward / backward	Moved forward and backward at once. Pass
7	Bluetooth	Press L / R	Turns left / right	Turned on the spot to the left / right. Pass
8	Bluetooth	Press S	Robot stops	Robot stopped immediately. Pass
9	Voice	Say the forward / back phrases	Moves forward / backward	Moved forward and backward. Pass
10	Voice	Say the left / right phrases with a free side	Scans, then turns	Servo scanned the side and the robot turned. Pass
11	Voice	Say a turn phrase with a blocked side (below 10 cm)	Robot stops without turning	Robot stayed in place. Pass
12	Mode switch	Turn on SW1, SW2 or SW3 alone	Only the selected mode runs	Each switch started its own mode. Pass

4.3 Ultrasonic Sensor Accuracy

The distance reported by the robot was compared with the distance read from a measuring tape for five obstacle distances (Table 4.2).

Actual distance (cm)	Measured distance (cm)	Error (cm)
5	5	0
10	10	0
20	19	1
50	49	1
100	98	2

The measured values agree with the tape to within 2 cm over the whole range, and the error grows slowly with distance (2 percent at 100 cm). This is accurate enough for the 12 cm stopping threshold used by the obstacle-avoidance mode.

4.4 Summary of Results

Criterion for success	Result
Stops and avoids an obstacle detected at 12 cm or less	Achieved (tests 2 to 5)
Responds to Bluetooth commands F, B, L, R, S without noticeable delay	Achieved (tests 6 to 8)
Responds to voice commands mapped to the same movements	Achieved (tests 9 to 11)
All four motors, the servo and the wireless and sensing parts work reliably from the 7.4 V pack	Achieved; operation became slow when the battery was low
Mode can be changed without re-uploading the code	Achieved with switches SW1 to SW3 (test 12)

4.5 Discussion

Obstacle avoidance. The decision rule of the firmware is simple and deterministic: forward when the path is clear, otherwise scan and turn toward the larger distance. It works best on flat surfaces with hard, flat obstacles facing the sensor. Soft materials such as cloth absorb the ultrasonic pulse, and obstacles with angled faces reflect it away from the receiver, so such objects may be detected late. The scan takes about 0.8 s per side because the servo needs time to reach each position, so the robot reacts more slowly at high speed.

Bluetooth control. Because each command is a single byte, the response is almost immediate within the normal range. The last received command stays active, so an explicit stop (S) is needed. Losing the Bluetooth link while the robot is moving will leave the last command running, which is a safety issue that could be solved with a time-out.

Voice control. The reliability of this mode depends on the speech recognition of the phone and on background noise, not on the robot. Checking the free distance before each turn adds a simple safety measure.

Practical issues. The Bluetooth module shares the serial port with USB, so its RX and TX wires must be removed during uploading. Low battery voltage causes slow motors and resets. Only one mode switch should be ON at a time, because the modes are checked one after the other in loop(). The distance messages printed in the obstacle mode also go out on the shared serial port.

4.6 Limitations

- No wheel encoders, so speed and distance travelled are not measured.
- A single ultrasonic sensor with a narrow beam; objects to the side of the beam can be missed.
- Bluetooth range of about 10 m and no cloud connectivity.
- The ENA/ENB jumpers keep the motors at full speed, so there is no software speed control.

Chapter 5

Conclusion

5.1 Conclusion

This project built a low-cost four-wheeled robot on an Arduino UNO that supports obstacle avoidance, Bluetooth control and voice control. An ultrasonic sensor on a scanning servo gives the robot a simple perception of its surroundings, the L298N module drives the four 60 RPM BO gear motors, and an HC-05 module connects the robot to a smartphone. The modular firmware lets each mode be selected independently with a switch, and the hardware stays simple and inexpensive. The report documented the circuit, the pin allocation, the firmware logic and the test procedure so that the robot can be reproduced and extended.

The work met its goals: the robot is able to detect an obstacle at 12 cm, choose a free direction, and accept wireless commands in two ways. It also showed the practical constraints of an embedded design, such as a shared serial port and battery limits.

5.2 Future Work

- Replace the HC-05 with an ESP32 or NodeMCU to add Wi-Fi and cloud (IoT) monitoring, for example with Blynk or MQTT.
- Connect the ENA and ENB pins of the L298N to PWM pins so that the robot speed can be controlled in software.
- Add a command time-out that stops the robot when the Bluetooth link is lost.
- Add wheel encoders and an IMU for speed control and straight-line driving.
- Use additional sensors (infrared, line-following) and a camera for vision-based navigation.

References

[1] A. Kolyshkin and S. Nazarovs, "Stability of slowly diverging flows in shallow water," Mathematical Modeling and Analysis, 2007. (Template reference; remove if not cited.)

[2] SriTu Hobby, "How to make a multi-function Arduino robot," 2021. [Online]. Available: https://srituhobby.com/how-to-make-a-multi-function-arduino-robot/

[3] Arduino, "Arduino UNO Rev3," Arduino Documentation. [Online]. Available: https://docs.arduino.cc

[4] Elec Freaks, "HC-SR04 Ultrasonic Ranging Module User Guide," datasheet.

[5] Texas Instruments, "L293x Quadruple Half-H Drivers," datasheet.